

instrumentation. The instrumentation tool adds the instrumentation code on top of the application's object code. A well-known example is the memory error detection tool 'Purify', available from www.ibm.com/software/awdtools/purifyplus/.

Static binary instrumentation

This instrumentation happens on the static binary executable. The instrumentation tool parses the executable, and adds the instrumentation code as machine-code instructions. Static binary instrumentation inserts additional code and data into an executable, and generates a persistent modified executable. Atom is a well-known static binary instrumentation framework available on the Alpha processor platform for the Tru-64 operating system, available at http://h30097.www3.hp.com/dtk/analysis/tools_ov.html. A number of popular tools, such as Pixie and Hiprof, have been built on top of it. However, since it is available only for the Alpha platform, it is not useful for today's popular general-purpose processors.

Dynamic binary instrumentation

Dynamic instrumentation tools insert additional code and data during execution, without making any permanent modifications to the executable. The main advantage of dynamic binary instrumentation is that it is applied at runtime, and hence can take advantage of the runtime information available, and use that information to optimise the inserted code. The second advantage is that since the instrumentation code is not persistent, different types of instrumentation can be applied for different runs on the same binary, without requiring re-compilation or re-linking.

Also, since the instrumentation is done at runtime, it facilitates the application of instrumentation code to a running application without having to stop and re-start the application. Once the required information has been collected, the tool facilitates the removal of the instrumentation code so that the executable runs as-is, without the instrumentation overhead from then onwards. This is not possible with statically instrumented executables.

With dynamic binary instrumentation, additional overhead is introduced, because the instrumentation tool must perform additional tasks such as parsing, disassembly, code generation, and making other decisions at runtime.

The biggest advantage of binary instrumentation tools, both static and dynamic, is that they do not require source code. This is important because in many cases, the applications can be legacy binaries, or may consist of third-party libraries where source code may not be readily available.

Dynamic Binary Instrumentation (DBI) frameworks

A dynamic binary instrumentation framework offers facilities which allow an analysis tool to add instrumentation dynamically at runtime on a binary executable with minimum effort. There are a number of popular DBI frameworks available, which are listed below.

Pin

Available from www.pintool.org, this is written along the lines of the Atom framework. Unlike Atom, Pin supports dynamic instrumentation of user applications, allowing arbitrary code written in C/C++ to be injected into arbitrary places in the executable. Pin provides a rich API that abstracts away the underlying instruction-set idiosyncrasies, and allows context information such as register contents to be passed to the injected code as parameters. Pin automatically saves and restores the registers that are overwritten by the injected code, so the application continues to work. Limited access to symbol and debug information is available as well. It supports Linux binary executables for Intel IA-32, Intel64 (64 bit x86), and Itanium (R) processors and Windows executables for IA-32 and Intel64. A good tutorial on the Pin framework and tools that can be developed on top of it is available at <http://www.cs.virginia.edu/kim/publicity/pin/tutorials/pin2-tutorial-2005.ppt>.

If you are looking for an interesting

academic project related to building a performance-profiling or instrumentation tool, PIN is definitely a framework to consider. However, unlike other open source DBI frameworks like Valgrind, only the source code for the tools is made available, while the source code for the instrumentation framework itself is not made available. Therefore, you cannot make enhancements to the DBI itself; you can only write tools on top of existing facilities.

DynamoRIO

This is another popular DBI platform, based on the Dynamo dynamic optimisation framework. Its source code and binaries are available from <http://dynamorio.org/>. It supports instrumentation of Windows and Linux IA-32 binaries. There are not as many popular tools on DynamoRIO as there are on Valgrind or Pin. Dr Memory is the popular memory error-checking tool built on top of DynamoRIO.

DynInst

Another popular instrumentation framework that supports both static and dynamic binary instrumentation is DynInst, which provides a machine-independent API that allows insertion of code to a static or dynamic binary. DynInst is available from www.dyninst.org. It is available for both Windows and Linux. A number of tools have been built on top of DynInst, such as SGI SpeedShop and Dynamic Probe Class Library (DPCL). The former is a performance profiler tool. DPCL is an object-based C++ class library that exports the necessary class interfaces for DBI.

Valgrind

To date, this is the most popular DBI framework in open source. A number of tools have been built on top of Valgrind, which include the popular memory error-checking tool MemCheck (www.valgrind.org). Valgrind supports heavy-weight dynamic binary instrumentation and complete shadow memory support. The comprehensive nature of Valgrind allows one to build very sophisticated tools, which may not be possible with DynamoRIO or DynInst. If you are a student looking to build an interesting tool using binary